

打卡功能网站开发文档

1. 项目概述

1.1 项目目标

开发一个功能完善的打卡网站,支持用户创建打卡计划、每日打卡、图片上传、日历展示等功能,包含完整的用户管理系统和数据分析能力。

1.2 核心功能

- 用户注册登录 (邮箱验证)
- 打卡计划创建与管理
- 每日打卡 (图片上传+备注)
- 日历视图展示打卡状态
- 补打卡功能

- 暗夜模式切换
- 用户行为埋点
- 单点登录支持
- 多端适配

2. 技术栈

2.1 前端技术栈

...

Vue 3.3+ (Composition API)

TypeScript 5.0+

Pinia (状态管理)

Vue Router 4

Element Plus 2.4+

Axios (HTTP 客户端)

Sass/SCSS (样式预处理器)

Day.js (日期处理)

Vite (构建工具)

VueUse (工具集)

...

2.2 后端技术栈

...

ASP.NET Core 8.0 Web API

Entity Framework Core 8.0

MySQL 8.0+

JWT Bearer Authentication

AutoMapper

Serilog (日志记录)

Quartz.NET (任务调度)

SignalR (实时通知)

FluentValidation

Swagger/OpenAPI

...

2.3 架构模式

三层架构 + DDD 领域驱动设计

...

表现层 (Presentation Layer) - Controllers

业务逻辑层 (Business Logic Layer) - Services

数据访问层 (Data Access Layer) -
Repositories

领域层 (Domain Layer) - Models, Interfaces

...

3. 数据库设计

3.1 用户表 (Users)

```
```sql
```

```
CREATE TABLE Users (
 Id INT PRIMARY KEY AUTO_INCREMENT,
 Email VARCHAR(255) UNIQUE NOT NULL,
 PasswordHash VARCHAR(512) NOT NULL,
 UserName VARCHAR(100),
 AvatarUrl VARCHAR(500),
 EmailConfirmed BIT DEFAULT 0,
 LastLoginAt DATETIME,
 LoginCount INT DEFAULT 0,
 IsBlacklisted BIT DEFAULT 0,
 BlacklistReason VARCHAR(500),
 CreatedAt DATETIME DEFAULT
CURRENT_TIMESTAMP,
 UpdatedAt DATETIME ON UPDATE
CURRENT_TIMESTAMP,
 DeletedAt DATETIME,
 IsDeleted BIT DEFAULT 0
);
```

## -- 索引

```
CREATE INDEX IX_Users_Email ON Users(Email);
CREATE INDEX IX_Users_IsBlacklisted ON
Users(IsBlacklisted);
```
```

3.2 打卡计划表 (CheckinPlans)

```
````sql  
CREATE TABLE CheckinPlans (
 Id INT PRIMARY KEY AUTO_INCREMENT,
 UserId INT NOT NULL,
 Title VARCHAR(200) NOT NULL,
 Description TEXT,
 StartDate DATE NOT NULL,
 EndDate DATE,
 DailyDeadline TIME DEFAULT '23:59:59',
 IsPublic BIT DEFAULT 1,
 IsContinuous BIT DEFAULT 1,
 ReminderSettings JSON,
```

```

 CreatedAt DATETIME DEFAULT
CURRENT_TIMESTAMP,
 UpdatedAt DATETIME ON UPDATE
CURRENT_TIMESTAMP,
 DeletedAt DATETIME,
 IsDeleted BIT DEFAULT 0,
 FOREIGN KEY (UserId) REFERENCES Users(Id)
ON DELETE CASCADE
);

```

## -- 索引

```

CREATE INDEX IX_CheckinPlans_UserId ON
CheckinPlans (UserId);

CREATE INDEX
IX_CheckinPlans_StartDate_EndDate ON
CheckinPlans (StartDate, EndDate);
```

```

3.3 打卡记录表 (CheckinRecords)

```

```sql

```

```

CREATE TABLE CheckinRecords (
 Id INT PRIMARY KEY AUTO_INCREMENT,
 PlanId INT NOT NULL,
 CheckinDate DATE NOT NULL,
 CheckinTime DATETIME NOT NULL,
 ImageUrl VARCHAR(500),
 ThumbnailUrl VARCHAR(500),
 Notes TEXT,

 IsMakeup BIT DEFAULT 0, -- 是否为补打卡
 Location JSON, -- {lat, lng, address}
 WeatherInfo JSON, -- {temperature,
weather, city}

 Status TINYINT DEFAULT 1, -- 1:正常 2:
补卡 3:漏卡

 CreatedAt DATETIME DEFAULT
CURRENT_TIMESTAMP,
 UpdatedAt DATETIME ON UPDATE
CURRENT_TIMESTAMP,
 DeletedAt DATETIME,
 IsDeleted BIT DEFAULT 0,

```



```

 UNIQUE KEY UK_Plan_Date (PlanId,
CheckinDate),

 FOREIGN KEY (PlanId) REFERENCES
CheckinPlans(Id) ON DELETE CASCADE
);

```

## -- 索引

```

CREATE INDEX IX_CheckinRecords_PlanId_Date
ON CheckinRecords(PlanId, CheckinDate);
CREATE INDEX IX_CheckinRecords_CheckinTime
ON CheckinRecords(CheckinTime);
```

```

3.4 操作日志表 (OperationLogs)

```

```sql
CREATE TABLE OperationLogs (
 Id INT PRIMARY KEY AUTO_INCREMENT,
 UserId INT,
 OperationType VARCHAR(50) NOT NULL, --

```

```

'Login', 'Checkin', 'CreatePlan'
 OperationDetail TEXT,
 IPAddress VARCHAR(45),
 UserAgent TEXT,
 DeviceType VARCHAR(50), -- 'Mobile',
'Tablet', 'Desktop'
 Platform VARCHAR(50), -- 'Web', 'iOS',
'Android'

 DurationMs INT, -- 操作耗时

 Success BIT DEFAULT 1,
 ErrorMessage TEXT,
 CreatedAt DATETIME DEFAULT
CURRENT_TIMESTAMP,
 INDEX IX_OperationLogs_UserId (UserId),
 INDEX IX_OperationLogs_CreatedAt
(CreatedAt),
 INDEX IX_OperationLogs_OperationType
(OperationType)
);

```

### 3.5 JWT 令牌黑名单表 (TokenBlacklists)

```
```sql
```

```
CREATE TABLE TokenBlacklists (  
    Id INT PRIMARY KEY AUTO_INCREMENT,  
    TokenId VARCHAR(100) NOT NULL,  
    UserId INT NOT NULL,  
    Expiry DATETIME NOT NULL,  
    Reason VARCHAR(200),  
    CreatedAt          DATETIME          DEFAULT  
CURRENT_TIMESTAMP,  
    INDEX              IX_TokenBlacklists_TokenId  
(TokenId),  
    INDEX IX_TokenBlacklists_UserId (UserId)  
);
```

```
```
```

## 4. 后端 API 设计

### 4.1 项目结构

...

Checkin.API/

└── Controllers/

└── Services/

└── Repositories/

└── Models/

  └── Entities/

  └── DTOs/

  └── ViewModels/

  └── Enums/

└── Interfaces/

└── Middlewares/

└── Filters/

└── Helpers/

└── Validators/

```
└── Program.cs
...

```

## 4.2 API 接口设计

### 4.2.1 认证相关

```
```csharp
// POST /api/auth/register
public async Task<IActionResult>
Register(RegisterRequest request)

// POST /api/auth/login
public async Task<IActionResult>
Login(LoginRequest request)

// POST /api/auth/refresh-token
public async Task<IActionResult>
RefreshToken(RefreshTokenRequest request)

```

```
// POST /api/auth/logout
public async Task<IActionResult> Logout()

// POST /api/auth/verify-email
public async Task<IActionResult>
VerifyEmail(string token)
```
```

#### 4.2.2 打卡计划管理

```
```csharp
// GET /api/plans
public async Task<IActionResult>
GetPlans([FromQuery] PlanQuery query)

// GET /api/plans/{id}
public async Task<IActionResult>
GetPlan(int id)

// POST /api/plans
```

```
public async Task<IActionResult>  
CreatePlan(CreatePlanRequest request)
```

```
// PUT /api/plans/{id}
```

```
public async Task<IActionResult>  
UpdatePlan(int id, UpdatePlanRequest  
request)
```

```
// DELETE /api/plans/{id}
```

```
public async Task<IActionResult>  
DeletePlan(int id)
```

```
// GET /api/plans/{id}/calendar
```

```
public async Task<IActionResult>  
GetPlanCalendar(int id, [FromQuery] int year,  
[FromQuery] int month)
```

```
````
```

### 4.2.3 打卡记录管理

```
````csharp
```

```
// POST /api/checkins  
public async Task<IActionResult>  
Checkin([FromForm] CheckinRequest request)
```

```
// POST /api/checkins/makeup  
public async Task<IActionResult>  
MakeupCheckin([FromForm]  
MakeupCheckinRequest request)
```

```
// GET /api/checkins/{planId}  
public async Task<IActionResult>  
GetCheckins(int planId, [FromQuery]  
DateTime startDate, [FromQuery] DateTime  
endDate)
```

```
// PUT /api/checkins/{id}  
public async Task<IActionResult>  
UpdateCheckin(int id, UpdateCheckinRequest  
request)  
...
```


4.2.4 统计与报表

```
```csharp
// GET /api/stats/overview
public async Task<IActionResult>
GetOverviewStats()

// GET /api/stats/plan/{id}
public async Task<IActionResult>
GetPlanStats(int id)

// GET /api/stats/calendar
public async Task<IActionResult>
GetCalendarStats([FromQuery] int year,
[FromQuery] int month)
```
```

5. 前端架构设计

5.1 项目结构

...

src/

| | |
|-----------------|-------------------|
| —— api/ | # API 请求封装 |
| —— assets/ | # 静态资源 |
| —— components/ | # 组件 |
| —— common/ | # 通用组件 |
| —— layout/ | # 布局组件 |
| —— checkin/ | # 打卡相关组件 |
| —— composables/ | # Composition API |
| —— router/ | # 路由配置 |
| —— stores/ | # Pinia 状态管理 |
| —— types/ | # TypeScript 类型定义 |
| —— utils/ | # 工具函数 |
| —— views/ | # 页面组件 |
| —— styles/ | # 样式文件 |

```

## 5.2 Pinia 状态管理设计

```
```typescript
// stores/user.ts
export const useUserStore =
defineStore('user', {
  state: () => ({
    userInfo: null as UserInfo | null,
    token: '',
    isDarkMode: false,
    preferences: {} as UserPreferences
  )),

  actions: {
    async login(credentials:
LoginCredentials) {
      // 登录逻辑
    },
  },
})
```

```

    async logout() {
        // 登出逻辑
    },

    toggleDarkMode() {
        this.isDarkMode = !this.isDarkMode
        localStorage.setItem('darkMode',
String(this.isDarkMode))
    }
},

persist: {
    key: 'user-storage',
    storage: localStorage
}
}))

// stores/checkin.ts
export const useCheckinStore =

```

```
defineStore('checkin', {
  state: () => ({
    currentPlan: null as CheckinPlan | null,
    calendarData: {} as Record<string,
CalendarDay>,
    checkinHistory: [] as CheckinRecord[]
  )),

  actions: {
    async fetchCalendarData(year: number,
month: number) {
      // 获取日历数据
    },

    async checkin(params: CheckinParams) {
      // 打卡逻辑
    }
  }
})
...

```

5.3 日历组件设计

```
```vue
<!-- components/checkin/Calendar.vue -->
<template>
 <div class="checkin-
calendar" :class="{ 'dark-mode' :
isDarkMode }">
 <div class="calendar-header">
 <div class="weekdays">
 <div v-for="day in
weekdays" :key="day" class="weekday">
 {{ day }}
 </div>
 </div>
 </div>
 <div class="calendar-body">
 <div
 v-for="day in days"
```

```

 :key="day.date"
 :class="[
 'calendar-day',
 day.status,
 {
 'today': day.isToday,
 'selected': day.isSelected
 }
]"
 @click="handleDayClick(day)"
 >

 <div class="day-
number">{{ day.day }}</div>

 <div v-if="day.hasCheckin"
class="checkin-indicator">

 <el-icon v-
if="day.isMakeup"><AlarmClock /></el-icon>

 <el-icon v-else><SuccessFilled
/></el-icon>

 </div>

 </div>

</div>

```

```
</template>
```

```
<script setup lang="ts">
```

```
import { ref, computed } from 'vue'
```

```
import { useCheckinStore } from
'@/stores/checkin'
```

```
const props = defineProps<{
```

```
 year: number
```

```
 month: number
```

```
const checkinStore = useCheckinStore()
```

```
// 周一至周日的中文显示
```

```
const weekdays = ['一', '二', '三', '四', '五', '六', '日']
```

```
// 计算日历天数
```



```

const days = computed(() => {
 // 日历计算逻辑
 return calendarDays.value.map(day => ({
 ...day,
 status: getDayStatus(day)
 })))
})

function getDayStatus(day: CalendarDay):
string {
 if (!day.hasCheckin && day.isPast
&& !day.isToday) return 'missed'
 if (day.isMakeup) return 'makeup'
 if (day.hasCheckin) return 'checked'
 return ''
}
</script>

```

```

<style scoped lang="scss">
.checkin-calendar {

```

```
&.dark-mode {
 --calendar-bg: #1a1a1a;
 --day-bg: #2d2d2d;
 --text-color: #ffffff;
}
```

```
.calendar-day {
 &.checked {
 background-color: var(--el-color-success-light-9);
 border-color: var(--el-color-success);
 }
}
```

```
&.makeup {
 background-color: var(--el-color-warning-light-9);
 border-color: var(--el-color-warning);
}
```

```
&.missed {
```

```

 background-color: var(--el-color-
danger-light-9);
 border-color: var(--el-color-danger);
 }

 &.today {
 border: 2px solid var(--el-color-
primary);
 }
}
</style>
` ``

```

## 5.4 打卡表单组件

```

` `` vue
<!-- components/checkin/CheckinForm.vue -->
<template>
 <el-form

```

```
 ref="formRef"
 :model="formData"
 :rules="rules"
 label-width="80px"
 @submit.prevent="handleSubmit"
 >
 <el-form-item label=" 打 卡 图 片 "
prop="images">
 <el-upload
 v-model:file-list="fileList"
 action="/api/upload"
 list-type="picture-card"
 :limit="5"
 :before-upload="beforeUpload"
 :on-success="handleUploadSuccess"
 :on-error="handleUploadError"
 >
 <el-icon><Plus /></el-icon>
 </el-upload>
 </el-form-item>
```

```
<el-form-item label=" 打 卡 备 注 "
prop="notes">
 <el-input
 v-model="formData.notes"
 type="textarea"
 :rows="3"
 placeholder=" 记 录 今 天 的 心 情 或 进
展..."
 maxlength="500"
 show-word-limit
 />
</el-form-item>
```

```
<el-form-item label=" 打 卡 位 置 " v-
if="enableLocation">
 <div class="location-info">
 <el-icon><Location /></el-icon>
 {{ locationInfo || ' 获 取 位 置
```

```
中...' }}
```

```
</div>
```

```
</el-form-item>
```

```
<el-form-item>
```

```
<el-button
```

```
type="primary"
```

```
:loading="submitting"
```

```
@click="handleSubmit"
```

```
>
```

```
{{ isMakeup ? '补打卡' : '立即打卡'
```

```
, }}
```

```
</el-button>
```

```
</el-form-item>
```

```
</el-form>
```

```
</template>
```

```
<script setup lang="ts">
```

```
import { ref, reactive, onMounted } from
'vue'
```

```
import { useCheckinStore } from
'@/stores/checkin'

import { uploadCheckin, uploadMakeupCheckin }
from '@api/checkin'

import { useUserStore } from '@stores/user'
```

```
const props = defineProps<{
 planId: number
 date?: string // 补打卡时传入日期
}>()
```

```
const formRef = ref()
const submitting = ref(false)
const fileList = ref([])
const locationInfo = ref('')
```

```
const formData = reactive({
 notes: '',
 images: [] as string[],
 location: null as LocationData | null
```

```
}}
```

```
// 获取地理位置
```

```
async function getLocation() {
 if (!navigator.geolocation) return

 try {
 const position = await new
 Promise<GeolocationPosition>((resolve,
 reject) => {

 navigator.geolocation.getCurrentPosition(re
 solve, reject, {
 enableHighAccuracy: true,
 timeout: 5000
 })
 })
 })
}
```

```
// 调用地理位置 API 获取地址信息
```

```
const { coords } = position
```



```
 const address = await
reverseGeocode(coords.latitude,
coords.longitude)
```

```
 locationInfo.value = address
 formData.location = {
 lat: coords.latitude,
 lng: coords.longitude,
 address
 }
 } catch (error) {
 console.error(' 获取地理位置失败:',
error)
 }
}
```

```
async function handleSubmit() {
 try {
 submitting.value = true
```

```
const data = {
 planId: props.planId,
 ...formData,
 checkinDate: props.date || new
Date().toISOString().split('T')[0]
}
```

```
if (props.date) {
 await uploadMakeupCheckin(data)
} else {
 await uploadCheckin(data)
}
```

```
ElMessage.success(' 打卡成功! ')
```

```
// 更新状态
```

```
} catch (error) {
 ElMessage.error(' 打卡失败')
} finally {
 submitting.value = false
```

```
 }
 }

 onMounted(() => {
 getLocation()
 })
</script>
```
```

6. 性能优化策略

6.1 前端性能优化

```
```typescript  

// 1. 图片懒加载和压缩

const useImageOptimization = () => {
 const compressImage = async (file: File,
 maxSize = 1024 * 1024): Promise<File> => {
 if (file.size <= maxSize) return file
```

```

return new Promise((resolve) => {
 const reader = new FileReader()
 reader.onload = (e) => {
 const img = new Image()
 img.onload = () => {
 const canvas =
document.createElement('canvas')
 const ctx =
canvas.getContext('2d')!

```

```

 let width = img.width
 let height = img.height
 const ratio = Math.sqrt(maxSize /
file.size)

```

```

 width *= ratio
 height *= ratio

```

```

 canvas.width = width
 canvas.height = height

```

```

 ctx.drawImage(img, 0, 0, width,
height)

 canvas.toBlob((blob) => {
 resolve(new File([blob!],
file.name, { type: 'image/jpeg' }))
 }, 'image/jpeg', 0.8)
 })
 img.src = e.target!.result as string
 }
 reader.readAsDataURL(file)
})
}
}

```

## // 2. 虚拟滚动日历

```

import { VirtualList } from 'vue-virtual-
scroll-list'

```

## // 3. 防抖节流

```
import { debounce, throttle } from 'lodash-es'
...

```

## 6.2 后端性能优化

```
```csharp
// 1. 缓存策略

public class CheckinService
{
    private readonly IMemoryCache _cache;
    private readonly ILogger<CheckinService>
_logger;

    public async Task<CalendarData>
GetCalendarDataAsync(int userId, int year,
int month)
    {
        var cacheKey =
$"calendar_{userId}_{year}_{month}";

```

```

        return await
_cache.GetOrCreateAsync(cacheKey, async
entry =>
    {

entry.AbsoluteExpirationRelativeToNow =
(TimeSpan.FromMinutes(5));

        // 数据库查询

        var data = await
_repository.GetCalendarDataAsync(userId,
year, month);

        return data;

    });
}
}

```

// 2. 数据库连接池配置

```

services.AddDbContext<AppDbContext>(options
=>

```

```

        options.UseMySQL(connectionString,
            new MySqlServerVersion(new
Version(8, 0, 28)),
            options =>
            {
                options.EnableRetryOnFailure(5,
                TimeSpan.FromSeconds(10), null);
                options.MaxBatchSize(100);
            }));
    }
}

```

7. 埋点系统设计

7.1 前端埋点 SDK

```

```typescript
// utils/tracking.ts
class TrackingSDK {
 private queue: TrackingEvent[] = []
 private isSending = false
}

```



```

private readonly batchSize = 10
private readonly flushInterval = 5000

constructor() {
 // 定时上报
 setInterval(() => this.flush(),
this.flushInterval)

 // 页面卸载前上报
 window.addEventListener('beforeunload',
() => this.flushSync())
}

track(event: string, properties:
Record<string, any> = {}) {
 const eventData: TrackingEvent = {
 event,
 properties: {
 ...properties,
 url: window.location.href,
 userAgent: navigator.userAgent,

```

```
 timestamp: Date.now(),
 sessionId: this.getSessionId(),
 userId: this.getUserId()
 }
}
```

```
this.queue.push(eventData)
```

```
// 达到批量大小立即上报
```

```
if (this.queue.length >= this.batchSize)
{
 this.flush()
}
}
```

```
private async flush() {
 if (this.isSending || this.queue.length
=== 0) return
```

```
 this.isSending = true
```

```
 const events = this.queue.splice(0,
this.batchSize)

 try {
 await fetch('/api/tracking', {
 method: 'POST',
 headers: { 'Content-Type':
'application/json' },
 body: JSON.stringify({ events }),
 keepalive: true // 确保在页面卸载时
也能发送
 })
 } catch (error) {
 // 失败重试

 this.queue.unshift(...events)
 } finally {
 this.isSending = false
 }
}
}
```

// Vue 插件

```
export const trackingPlugin = {
 install(app: App, options: any) {
 const sdk = new TrackingSDK()
 app.config.globalProperties.$track =
 sdk.track.bind(sdk)
```

// 自动追踪路由

```
 app.router?.afterEach((to, from) => {
 sdk.track('page_view', {
 page: to.path,
 referrer: from.path
 })
 })
 }
}
```

## 7.2 关键埋点事件

```
```typescript
// 用户行为埋点
const trackUserBehavior = () => {
  // 打卡相关
  track('checkin_start', { plan_id: planId })
  track('checkin_complete', {
    plan_id: planId,
    has_image: hasImage,
    note_length: note.length,
    duration: duration
  })
  track('checkin_fail', { plan_id: planId,
error: error.message })

  // 页面交互
  track('button_click', { button_id:
buttonId, page: currentPage })
}
```

```
    track('form_submit', { form_id: formId,  
success: isSuccess })
```

// 性能指标

```
track('page_load', {  
    page: pageName,  
    load_time: performance.now(),  
    ttfb: performance.timing.responseStart  
- performance.timing.requestStart  
})  
}  
...
```

8. 暗夜模式实现

8.1 SCSS 主题配置

```
```scss  
// styles/_variables.scss
:root {
```

// 明亮主题

```
--bg-color: #ffffff;
--text-color: #303133;
--border-color: #dcdfe6;
--card-bg: #f5f7fa;
--primary-color: #409eff;
```

// 组件颜色

```
--calendar-day-bg: #ffffff;
--calendar-border: #ebeef5;
--success-color: #67c23a;
--warning-color: #e6a23c;
--danger-color: #f56c6c;
```

}

.dark-mode {

// 暗夜主题

```
--bg-color: #1a1a1a;
--text-color: #e5eaf3;
```

```
--border-color: #4c4d4f;
--card-bg: #242424;
--primary-color: #409eff;
```

## // 组件颜色

```
--calendar-day-bg: #2d2d2d;
--calendar-border: #3d3d3d;
--success-color: #85ce61;
--warning-color: #ebb563;
--danger-color: #f78989;
}
```

## // 使用变量

```
.app-container {
 background-color: var(--bg-color);
 color: var(--text-color);
 transition: background-color 0.3s, color
0.3s;
}
```



```
.calendar-day {
 background-color: var(--calendar-day-bg);
 border: 1px solid var(--calendar-border);
}
...
```

## 8.2 Vue 组件中的主题切换

```
```vue  
<!-- components/layout/ThemeToggle.vue -->  
<template>  
  <el-button  
    :icon="isDarkMode ? 'Sunny' : 'Moon'"  
    circle  
    @click="toggleTheme"  
  />  
</template>  
  
<script setup lang="ts">  
import { useUserStore } from '@stores/user'
```

```
const userStore = useUserStore()
const isDarkMode = computed(() =>
userStore.isDarkMode)
```

```
function toggleTheme() {
  userStore.toggleDarkMode()
```

```
  if (userStore.isDarkMode) {
```

```
    document.documentElement.classList.add('dark-mode')
  } else {
```

```
    document.documentElement.classList.remove('dark-mode')
  }
```

```
}
```

```
}
```

```
// 初始化主题
```

```
onMounted(() => {
```

```

        const savedTheme =
localStorage.getItem('darkMode')
        if (savedTheme === 'true') {

document.documentElement.classList.add('dark-mode')

        userStore.isDarkMode = true
    }
})
</script>
```

```

## 9. 单点登录实现

### 9.1 JWT 认证配置

```

```csharp
// Program.cs

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)

```

```

        .AddJwtBearer(options =>
        {
            options.TokenValidationParameters =
new TokenValidationParameters
            {
                ValidateIssuer = true,
                ValidateAudience = true,
                ValidateLifetime = true,
                ValidateIssuerSigningKey = true,
                ValidIssuer =
builder.Configuration["Jwt:Issuer"],
                ValidAudience =
builder.Configuration["Jwt:Audience"],
                IssuerSigningKey = new
SymmetricSecurityKey(
Encoding.UTF8.GetBytes(builder.Configuration
n["Jwt:Key"]!))
            };

```

// SignalR 支持

```

        options.Events = new
JwtBearerEvents
    {
        OnMessageReceived = context =>
        {
            var accessToken =
context.Request.Query["access_token"];
            var path =
context.HttpContext.Request.Path;
            if
(!string.IsNullOrEmpty(accessToken) &&
path.StartsWithSegments("/hubs"))
            {
                context.Token =
accessToken;
            }
            return Task.CompletedTask;
        }
    };
});

```

// JWT 刷新 Token 机制

```
builder.Services.AddScoped<ITokenService,  
TokenService>();  
...
```

9.2 Token 服务实现

```
```csharp  
public class
TokenService : ITokenService {
 private
 readonly IConfiguration _configuration;
 private readonly AppDbContext _context;
 public async Task<AuthResult>
GenerateTokensAsync(User user)
 {
 var accessToken =
GenerateAccessToken(user);
 var
refreshToken = GenerateRefreshToken();
 // 保存刷新 Token 到数据库
 var
refreshTokenEntity = new RefreshToken
 {
 UserId = user.Id,
 Token = refreshToken, Expires =
DateTime.UtcNow.AddDays(7),
 CreatedAt = DateTime.UtcNow
 };
 await
```

```

_context.RefreshTokens.AddAsync(refreshTokenEntity);
_context.SaveChangesAsync();
return new AuthResult
{
 AccessToken = accessToken,
 RefreshToken = refreshToken,
 ExpiresIn = 3600 // 1 小时
};
}

public async Task<AuthResult>
RefreshTokenAsync(string refreshToken)
{
 var tokenEntity = await
_context.RefreshTokens.Include(
t =>
t.User).FirstOrDefaultAsync(t
=> t.Token == refreshToken && t.Expires >
DateTime.UtcNow);
 if (tokenEntity ==
null) throw new SecurityException("Invalid
refresh token"); // 删除旧的刷新
Token
_context.RefreshTokens.Remove(tokenEntity);
// 生成新的 Token 对 return await
GenerateTokensAsync(tokenEntity.User);
}
}

10. 部署配置 10.1 Docker 配置
dockerfile# 前端 DockerfileFROM node:18-

```

```

alpine as buildWORKDIR /appCOPY
package*.json ./RUN npm ciCOPY . .RUN npm
run buildFROM nginx:alpineCOPY --from=build
/app/dist /usr/share/nginx/htmlCOPY
nginx.conf /etc/nginx/nginx.confEXPOSE
80CMD ["nginx", "-g", "daemon off;"]# 后端
DockerfileFROM
mcr.microsoft.com/dotnet/aspnet:8.0 AS
baseWORKDIR /appEXPOSE 80EXPOSE 443FROM
mcr.microsoft.com/dotnet/sdk:8.0 AS
buildWORKDIR /srcCOPY
["Checkin.API/Checkin.API.csproj",
"Checkin.API/"]RUN dotnet restore
"Checkin.API/Checkin.API.csproj"COPY . .RUN
dotnet build -c Release -o /app/buildFROM
build AS publishRUN dotnet publish -c
Release -o /app/publishFROM base AS
finalWORKDIR /appCOPY --from=publish
/app/publish .ENTRYPOINT ["dotnet",
"Checkin.API.dll"]```10.2 Nginx 配 置
```nginx#      nginx.confupstream      backend
{      server api:5000;}server {      listen 80;
server_name yourdomain.com;      # 前端静态文

```



```

件          location / {          root
/usr/share/nginx/html;          index
index.html;          try_files $uri $uri/
/index.html;          # 缓存策略
location ~*
\. (js|css|png|jpg|jpeg|gif|ico|svg)$ {
expires 1y;          add_header Cache-
Control "public, immutable";          } }
# API 代理          location /api
{          proxy_pass http://backend;
proxy_http_version          1.1;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection keep-alive;
proxy_set_header Host $host;
proxy_cache_bypass $http_upgrade;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
# 超时设置          proxy_connect_timeout 30s;
proxy_send_timeout          30s;
proxy_read_timeout 30s;          } # WebSocket
支持          location /hubs {          proxy_pass

```

```

http://backend; proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection
"upgrade";    }}```

```

11. 监控与日志 11.1 前端

```

错误 监控 ```typescript//
utils/errorHandler.tsclass ErrorHandler
{
  init() { // Vue 错误处理
    app.config.errorHandler = (err, instance,
    info) => { this.trackVueError(err,
    info) } // 全局错误处理
    window.addEventListener('error', (event) =>
    { this.trackJsError(event.error ||
    event.message) }) // Promise 错误处理
    window.addEventListener('unhandledrejection',
    (event) =>
    { this.trackPromiseError(event.reason)
    }) // 资源加载错误
    window.addEventListener('error', (event) =>
    { if (event.target instanceof
    HTMLInputElement)
    { this.trackResourceError(event)
    }, true) } private

```

```
trackJsError(error: Error)
{
    this.report({
        type: 'js_error',
        message: error.message,
        stack: error.stack,
        url: window.location.href,
        timestamp: Date.now()
    })
}
```

11.2 后端

日志配置

```
Program.csbuilder.Host.UseSerilog((context,
    config)
```

```
=>{
    config.ReadFrom.Configuration(context.
        Configuration)
        .Enrich.FromLogContext()
        .WriteTo.Console(outputTemplate:
            "[{Timestamp:HH:mm:ss}
            {Level:u3}]
            {Message:l} {NewLine} {Exception} ")
        .WriteTo.File("logs/app-.txt",
            rollingInterval: RollingInterval.Day,
            retainedFileCountLimit:
                30)
        .WriteTo.Seq(serverUrl:
            context.Configuration["Seq:ServerUrl"]!);
});
```

12. 安全措施

12.1 安全中间件

```
SecurityHeadersMiddleware.cs
public class
SecurityHeadersMiddleware{
    public async
```

```

Task InvokeAsync(HttpContext context)
{
    context.Response.Headers.Add("X-
Frame-Options", "DENY");
context.Response.Headers.Add("X-Content-
Type-Options", "nosniff");
context.Response.Headers.Add("X-XSS-
Protection", "1; mode=block");
context.Response.Headers.Add("Referrer-
Policy", "strict-origin-when-cross-
origin");
context.Response.Headers.Add("Content-
Security-Policy", "default-src
'self'; img-src 'self' data: https:");
await _next(context);    }}// 请求限流
builder.Services.AddRateLimiter(options
=>{
    options.GlobalLimiter =
PartitionedRateLimiter.Create<HttpContext,
string>(context =>
RateLimitPartition.GetFixedWindowLimiter(
    partitionKey:
context.User.Identity?.Name ??
context.Request.Headers.Host.ToString(),
factory: partition => new
FixedWindowRateLimiterOptions

```

```
{
    AutoReplenishment = true,
    PermitLimit        =        100,
    QueueLimit = 0,
    Window =
    TimeSpan.FromMinutes(1)
    }));});`
```

``13. 开发约定 13.1 代码规范 · 使用 ESLint + Prettier 进行代码格式化 · TypeScript 严格模式开启 · 组件使用 Composition API + <script setup> · 统一使用 Element Plus 组件库 · API 请求统一使用 Axios 拦截器处理 13.2 Git 工作流

```master - 生产分支 develop - 开发分支 feature/\* - 功能分支 hotfix/\* - 热修复分支 release/\* - 发布分支``13.3 提交规范

```feat: 新功能 fix: 修复 bug docs: 文档更新 style: 代码格式 refactor: 重构 test: 测试 chore: 构建过程或辅助工具的变动``---这个开发文档涵盖了项目的完整技术细节和实现方案。在实际开发过程中,可以根据具体需求进行调整和优化。建议采用敏捷开发模式,分阶段迭代开发,确保项目质量和开发进度。

